

Behavioral Round Prep – HPE Project

Based on real experiences from the Authorization Service / Payment Gateway project

Format to follow: STAR – Situation → Task → Action → Result

Keep each answer under **2 minutes** when speaking.

1. "Tell me about a challenge you faced and how you overcame it."

Experience: EC2 IP changing on every restart

S: Every time we shut down our AWS instances to save credits and restarted them, the IP address would change. This broke all our service connections – Kafka brokers couldn't be reached, and the entire Kubernetes cluster had to be manually reconfigured each time.

T: We needed a stable setup so we could restart without losing hours to reconfiguration before every test.

A: I researched the issue and found that AWS provides Elastic IPs – static IP addresses that stay attached to an instance even after a restart. We assigned one to our master node so its address never changed.

R: After that, restarts were clean. No more manual fixes each time, and we could focus our time on actual development and testing instead of setup.

2. "Tell me about a time you had to learn something quickly."

Experience: First time doing real development

S: My background for two years had been purely competitive programming. I had never worked with Kafka, Docker, Kubernetes, or cloud platforms before joining this program. Everything was new on day one.

T: I had to build and deploy a working backend service from scratch within a few weeks as part of a larger system.

A: I broke it down into smaller steps — first understood the overall system design, then focused on my specific service, and then learned the deployment side. I used documentation, asked teammates when stuck, and learned by building rather than just reading.

R: I successfully built and deployed the Authorization Service in C++, something I had zero prior experience with. It was the steepest learning curve I've had, but also the most rewarding outcome.

3. "Tell me about a time you worked in a team."

Experience: Different teammates owning different services

S: Our project had multiple services — API Gateway, Authorization, Clearing, and Settlement. Each teammate owned one area, but all services had to work together as one system.

T: My service had to correctly receive events from the API Gateway and publish results that the Clearing Service could pick up. Getting that coordination right was critical.

A: We had regular discussions to agree on event formats and field names so all services could communicate without breaking each other. When a teammate updated the database schema by adding a balance field, I updated my service to handle it without being asked.

R: The system worked end-to-end because we stayed aligned. It taught me that clear communication and coordination matter as much as the code itself.

4. "Tell me about a time you debugged a difficult problem."

Experience: Cloudflare blocking our own load test traffic

S: We set up Cloudflare to protect our system, but when we ran load tests, most of the test traffic was being blocked. Our performance results were completely wrong and we couldn't figure out why at first.

T: We had to identify why the test was failing and fix it without spending money on premium plans.

A: After investigating, we realized Cloudflare's free plan was treating our own high-volume test traffic as an attack. We couldn't whitelist our test machine's IP address on the free tier. We decided to replace Cloudflare with NGINX and ModSecurity, which we could fully configure ourselves inside our own cluster.

R: After switching, we had complete control over traffic rules, whitelisted our test machines, and our load tests ran correctly. We also got better visibility into what was happening with the traffic.

5. "Tell me about a time you had to adapt to change."

Experience: Teammate updated the database schema mid-project

S: Mid-way through the project, my teammate added a new feature — a balance check — by adding a new table to the database. This changed how my service needed to query the database to work correctly.

T: I had to integrate this change into my service without breaking any of the existing validation logic that was already working.

A: I reviewed the changes, understood what was added, and updated the SQL query in my service to include the new table. I also added the balance validation logic — checking if the user had enough balance before authorizing the payment.

R: The change was integrated smoothly without any disruption. The service became more complete because it now covered one more real-world validation scenario.

6. "Tell me about a time you worked under constraints."

Experience: AWS Free Tier budget limitation

S: Our entire infrastructure ran on AWS, but we were on the Free Tier which gave us limited compute credits. We couldn't keep our instances running 24/7 or we'd exhaust the credits quickly.

T: We had to plan our development and testing around this constraint without slowing the project down too much.

A: We established a routine — spin up the cluster, do testing or development, then shut it down immediately after. We also solved the IP issue with Elastic IPs so restarts weren't painful. We planned our load testing windows in advance to make the most of each session.

R: We managed to run all our load tests and complete the project within the Free Tier budget. It also taught me to think practically about resource costs, which is a real concern in any production environment.

7. "Tell me about a time you made a technical decision and stood by it."

Experience: Choosing C++ over Java for the service

S: When I started building the Authorization Service, I had to decide what technology to use. Most teams in similar projects default to Java or Spring Boot.

T: Our target was 10,000 requests per second, and I needed a language that could handle that without becoming a bottleneck.

A: I chose C++ because of its low latency and the absence of garbage collection pauses that Java has. I built a custom thread pool and database connection pool to handle high concurrency. It was harder to write than Java, but I was confident it was the right call for the performance requirement.

R: The service met the performance target. It also gave me a real appreciation for why language choice matters in systems that need to handle high load – something I had only theoretically understood from competitive programming before.

8. "Tell me about a time you failed or made a mistake."

Experience: Getting stuck on the "I'll start dev later" loop

S: For two years, I kept telling myself I'd start development once I hit a certain rank in competitive programming – first Pupil, then Specialist. Even after reaching Specialist, I kept delaying.

T: I wanted to be "ready" before starting, but kept moving the goalpost.

A: I didn't actually fix this myself – I got pushed into it by being selected for the HPE program. But once I was in it, I stopped waiting and just started building. I accepted that I would make mistakes and learned as I went.

R: I built a complete backend service, learned cloud deployment, and worked in a real team – all things I'd been postponing. The lesson was that waiting until you feel "ready" is itself the mistake. You get ready by doing.



Quick One-Liners for Common Follow-ups

Follow-up	Answer
-----------	--------

Follow-up	Answer
"What would you do differently?"	"Started development earlier instead of waiting until I felt ready."
"How did the team handle disagreements?"	"We discussed design decisions openly – things like which tech to use were debated and decided together."
"What was your biggest takeaway?"	"Real engineering is a different kind of hard than competitive programming. It's not just solving a problem – it's making something that works reliably for others."
"How do you handle not knowing something?"	"I break it down, start with what I do know, and fill gaps through docs, teammates, or just trying things."
"Describe yourself in one line."	"A competitive programmer who finally started building things – and realized both skills complement each other."